

Connected Vehicle Intelligence Hackathon

Solution Document — WORKING DRAFT (complete bracketed items before submission)

Submission Format: PDF (export of this document)

To be Submitted by: [ADD TEAM NAME]

Team Members & Roles: [ADD EACH MEMBER: NAME — ROLE — EMAIL]

Problem Space Chosen: Fleet Intelligence Platform — connected-vehicle fleet operations

Repository URL: <https://github.com/markapurammarthanda12/Fleet-Intelligence-Platform>

Demo Video URL (≤ 5 min): [ADD VIDEO URL AFTER RECORDING]

Date of Submission: [ADD SUBMISSION DATE]

Table of Contents

- [1. Executive Summary](#)
- [2. Problem Statement & Validation](#)
 - [2.1 Problem Statement](#)
 - [2.2 Evidence & Validation](#)
 - [2.3 Impact & Success Metrics](#)
- [3. Solution Description](#)
 - [3.1 Solution Overview & User Journey](#)
 - [3.2 Key Value Proposition](#)
 - [3.3 Innovative Ideas](#)
- [4. Feature List](#)
- [5. Solution Architecture \(High-Level Design\)](#)
 - [5.1 Architecture Overview](#)
 - [5.2 Technology Stack & Justification](#)
 - [5.3 Data Architecture](#)
 - [5.4 Deployment View](#)
- [6. Low-Level Design](#)
 - [6.1 Layering & Separation of Concerns](#)
 - [6.2 Design Principles Applied](#)
 - [6.3 Design Patterns Used](#)
 - [6.4 Interfaces, Contracts & Runtime Flows](#)
 - [6.5 Algorithms & Data Structures](#)
- [7. Non-Functional Requirements & Performance Benchmarks](#)
- [8. Security & Compliance](#)
- [9. Test Strategy](#)
- [10. Observability](#)
- [11. AI / ML Component \(if used\)](#)
- [12. Architecture Decisions, Risks & Future Enhancements](#)
- [13. Demo Video \(5 Minutes Maximum\)](#)
- [14. Repository Checklist](#)
- [15. Conclusion](#)
- [16. Declarations](#)
- [17. Appendix \(if any\)](#)

1. Executive Summary

Brief overview (half a page maximum) of:

- The problem you chose and who it affects
- What your solution does, in one or two sentences
- Key results achieved (throughput, latency, impact numbers)
- Unique features, innovations or optimisations you implemented

Fleet managers need a timely, explainable view of vehicle activity and operational risk. The local demo sends synthetic telemetry through a Spring Boot API and Kafka, stores operational state and alerts in PostgreSQL, aggregates history in ClickHouse, and presents results in a React dashboard. At the 2026-10-02 database check, the demo contained a 100,000-vehicle overview, 980,932 telemetry events, and 11,517 alert records across two tenants. One prior 37-hour Reports sample took 0.16 seconds after hourly rollups; the current populated analytics sample took 0.690 seconds for about 27 hours. A warm-cache alert query plan comparison was 12.910 ms with a forced scan/sort versus 0.100 ms with the V7 index. The simulator is configured for 5 events/second; 100,000 events/second, burst tolerance, p95/p99, end-to-end freshness, and customer impact remain unmeasured. Fuel and CO2 are estimates.

2. Problem Statement & Validation

2.1 Problem Statement

- **Format:** “[User] needs a way to [job to be done] because [pain], which today costs [measurable impact].”
- **Primary User:** Fleet manager, dealer, lender, driver or OEM — name one
- **Secondary Stakeholders:** Others affected by the problem

Primary user: fleet operations manager responsible for vehicle health, operational alerts, utilization, and costs. Need: a consolidated way to identify vehicles needing attention and understand why. Problem hypothesis: siloed or delayed vehicle signals make timely intervention and cost attribution harder. A measurable customer cost baseline has not been collected; no fleet interviews or production observations were performed. The official brief supplies the synthetic scale scenario, while this prototype exercises a representative workflow. Cost reduction and safety benefits are hypotheses, not validated outcomes.

2.2 Evidence & Validation

- **Facts vs Assumptions:** Separate what you know from what you assume
- **Validation Method:** Published study, public dataset, simulation, or interviews
- **Existing Alternatives:** Aftermarket dongles, OEM portals, Motorq Fuse, other telematics tools — and why they fall short

Evidence / Assumption	Source or Method	What It Shows	Confidence
Synthetic fleet scale	Simulator source and automated tests	100,000 unique synthetic vehicle IDs; not 100K events/sec	High for catalog count
Idle-fuel estimate	Rule configuration / synthetic events	Illustrative 1.5 L/hour assumption; requires calibration	Low / assumption

2.3 Impact & Success Metrics

Metric	Baseline Today	Target	How Measured / Estimated
Synthetic catalog size	Not applicable	100,000 vehicle IDs	Generator test; not throughput
Idle fuel / emissions impact	Not measured on real fleet	Explainable estimate only	Synthetic rule; actual savings unknown

- **Scale of Impact:** Who benefits, by how much, at 10K vs 100K vehicles
- **Wider Impact:** Safety, cost, environment or compliance, where relevant

The simulator can generate 10,000- and 100,000-vehicle catalogs for repeatable demonstrations. The checked local overview represents 100,000 vehicles; current-event freshness, 100,000 events/second capacity, and 3x bursts are not established at either scale. The official case study estimates one event per second per vehicle and about 1 KB per event; that is a workload assumption, not customer data or a measured baseline. Potential benefits include earlier attention to idling and visibility into estimated fuel use. Actual savings, emissions reductions, safety improvement, and fleet-manager time saved need field baselines and a controlled evaluation.

3. Solution Description

3.1 Solution Overview & User Journey

- **What It Does:** Plain-language description of the solution
- **User Journey:** Step by step, e.g. vehicle event → detection → alert → action → outcome
- **Screenshot:** At least one screenshot of the working product

The following Overview and Alerts screenshots were captured from the local synthetic fleet demo. Capture details and limits are in [evidences/2026-10-01/screenshot-capture.md](#).

Working product screenshots

The screenshots below show the local Docker demo using synthetic telemetry. Counts are a changing snapshot and are not performance measurements.

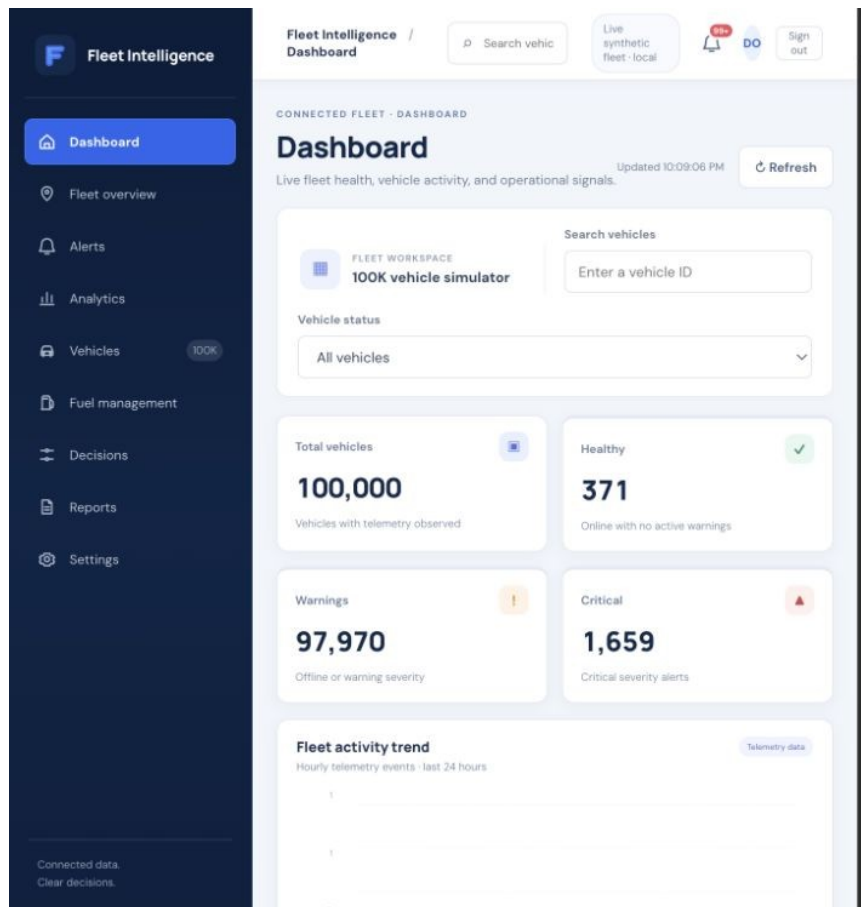


Figure 1. Dashboard overview for the synthetic 100,000-vehicle fleet. At capture: 371 healthy, 97,970 warnings, and 1,659 critical. Captured 1 October 2026 at 10:09 PM IST; browser viewport 714 × 746 pixels.

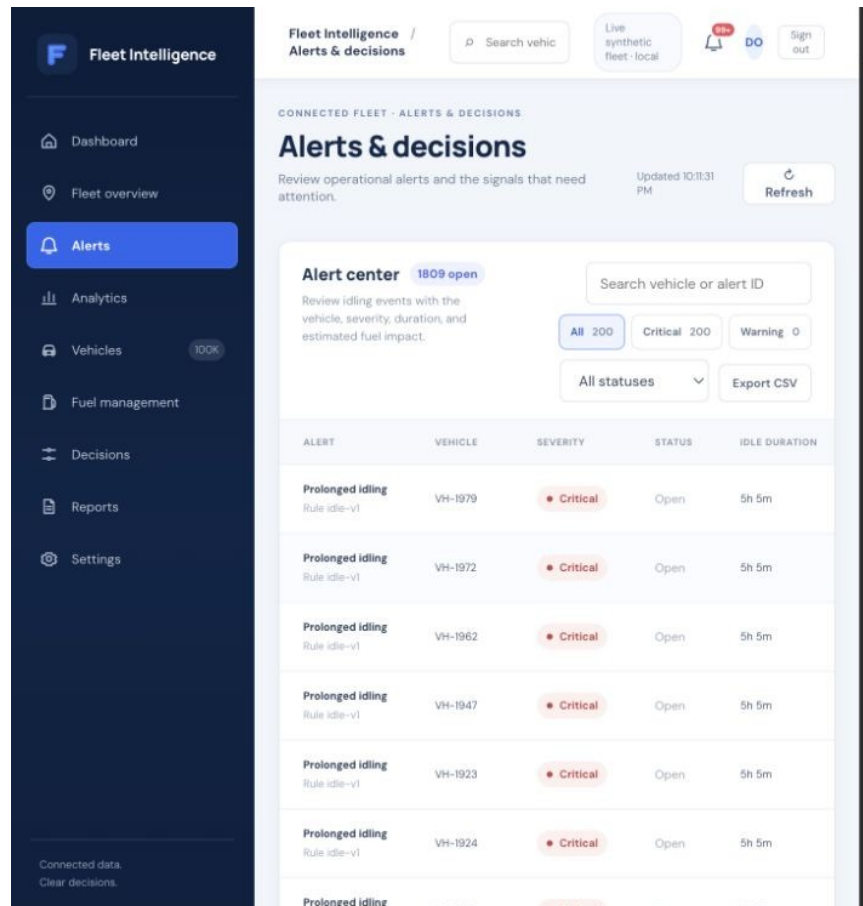


Figure 2. Alerts view at capture: 1,809 open alerts overall; the table loaded 200 alerts, all critical, and the loaded results contained 0 warnings. This is a paginated snapshot. Captured 1 October 2026 at 10:11 PM IST; browser viewport 714 × 746 pixels.

3.2 Key Value Proposition

- **Customer Job:** What the user is trying to get done
- **Pain Relieved:** What gets easier, cheaper or safer
- **Gain Created:** New capability the user did not have
- **Differentiation:** Why your solution, not the alternatives

Customer job: see fleet status and decide which vehicle or alert needs attention. Pain relieved: operators can inspect a consistent fleet view and trace a prolonged-idle alert to telemetry and an estimated fuel impact. Gain: repeatable synthetic data, live dashboard updates, replayable Kafka ingress, and a separate analytical history path. Differentiation in this submission is the explainable end-to-end demonstration and 100K synthetic catalog, not a claim of production differentiation validated against competitors.

3.3 Innovative Ideas

- **Up to Three Ideas:** A technique, data insight, user experience or business model
- **What Is New:** Compared against prior work you studied
- **Evidence:** Proof that each idea works

1) Reproducible fleet simulation: seeded generation makes the synthetic catalog repeatable; automated tests verify 100,000 unique IDs. 2) Separate operational and analytical paths: Kafka feeds PostgreSQL-backed fleet state and ClickHouse hourly aggregates, keeping report scans apart from core transactional reads. 3) Explainable, data-driven demonstration: event-time idle rules produce persistent warning/critical alerts, while a user-triggered synthetic refuelling batch updates fuel totals. Evidence is the simulator test suite, backend integration suite, working dashboard screenshots, and local runtime/query-plan notes. No product-versus-competitor benchmark or customer impact study has been performed.

4. Feature List

Every feature must be traceable to code. Priority uses MoSCoW (Must / Should / Could / Won't). Status: Done / Partial / Planned. Give the demo video timestamp for Done features.

ID	Feature	User Story	Priority	Status	Code Path	Video
F-01	Synthetic fleet overview	As a fleet manager I can review fleet health and vehicle status	Must	Done (demo)	frontend/src/App.tsx; GET /v1/fleet/overview	[ADD]
F-02	Telemetry ingestion and idling alerts	As a fleet manager I can inspect and acknowledge explainable idle alerts	Must	Partial (demo path)	backend/src/main/java/com/fleetintelligence/telemetry; frontend/src/App.tsx	[ADD]
F-03	Analytics and simulated refuelling	As a fleet manager I can view historical estimates and trigger a synthetic fuel update	Should	Partial (synthetic)	backend/src/main/java/com/fleetintelligence/analytics; backend/src/main/java/com/fleetintelligence/telemetry/FuelController.java	[ADD]

5. Solution Architecture (High-Level Design)

5.1 Architecture Overview

- **System Context Diagram:** C4 level 1 — your system, users and external systems (OEM clouds, identity provider, LLM API, data warehouse)

- **Container Diagram:** C4 level 2 — every service, data store and broker, with the protocol on each arrow (MQTT, HTTPS, gRPC, Kafka)
- **Data Flow:** Path of one telemetry event from vehicle to insight, with expected latency per hop

Appendix A includes the system context, containers, event flow, relational ER, deployment view, service layers, and two sequences. The local demo is a single-machine Compose topology; the proposed high-availability/cloud portions are design-only. Ingest returns 202 after Kafka acknowledgement, followed by asynchronous PostgreSQL/ClickHouse consumers; per-hop timings have not been measured. See docs/ARCHITECTURE.md and docs/DIAGRAMS.md for architecture descriptions and source diagrams.

5.2 Technology Stack & Justification

Layer	Choice	Why This, and What You Rejected
Ingestion / Messaging	Spring Boot REST + Apache Kafka	Kafka retains/replays keyed telemetry; local broker is single-node, so HA target is unmet
Stream / Batch Processing	Kafka consumer + deterministic event-time rule; ClickHouse hourly analytics	Separate operational processing from analytical scans; no 100K events/sec proof
Relational / NoSQL / Cache / Search / Vector	PostgreSQL + ClickHouse; no cache/vector store	Transactional tenant/vehicle state vs columnar historical analytics; omit vector absent use case
Backend / Frontend	Java Spring Boot; React + TypeScript + Vite	Existing team stack, API separation, interactive dashboard
ML / AI	Not used	Explainable rules meet current demonstration need; avoid unsupported model claims
Infrastructure / CI-CD / Observability	Docker Compose + GitHub Actions	Repeatable local stack/build; cloud HA and complete production observability remain pending

5.3 Data Architecture

- **ER Diagram:** Relational core normalised to 3NF; list any deliberate denormalisation and why
- **Polyglot Map:** Which data lives in which store, with its CAP choice (CP or AP)
- **Capacity Estimate:** Events/sec, bytes/event, daily and yearly volume, partition keys, hot / warm / cold retention
- **Query Optimisation:** Top three queries with EXPLAIN ANALYZE before and after

Query / sample	Before or baseline plan	After / current plan	Change / interpretation
37-hour ClickHouse Reports request (historical local sample)	21.3 s raw-history path	0.16 s after hourly rollups	One earlier local request; full hours read rollups and partial boundaries raw data. Not a load test.
PostgreSQL Alerts list, 100 rows (same query and DB snapshot)	12.910 ms forced sequential scan + top-N sort	0.100 ms V7 expression -index scan	One warm-cache plan comparison. Baseline scan was forced after V7 existed; it is not a historical pre-deployment run.
Fleet overview over 100,000 current states	Not captured before	82.143 ms current plan	One EXPLAIN ANALYZE sample; full tenant-state aggregation. No before/after change.
Vehicle page, newest 200 states	Not captured before	0.461 ms current plan	One warm-cache EXPLAIN ANALYZE sample using tenant/latest and open-alert indexes. No before/after change.

5.4 Deployment View

- **Deployment Diagram:** Clusters, namespaces, replicas, autoscaling, networking, secrets
- **Cloud-Agnostic Approach:** How the same deployment runs on a second cloud or locally

Appendix A distinguishes the running local Compose topology from a proposed multi-node deployment. Local Compose is the only verified deployment. No Helm/Kubernetes/Terraform pack, cloud account deployment, multi-zone cluster, or second-cloud deployment has been verified; no claim of portability without code changes or 99.9% availability is made.

6. Low-Level Design

6.1 Layering & Separation of Concerns

- **Architecture Style:** Layered, hexagonal (ports and adapters) or clean architecture — justify
- **Layer diagram:** Appendix A shows intended API, application, domain, and infrastructure responsibilities. Repository tree:

```
backend/src/main/java/com/fleetintelligence/{analytics,security,telemetry};
backend/src/main/resources/{db/migration,clickhouse}; frontend/src; src/fleetpulse; scripts;
tests; docs; infra/clickhouse.
```

The current TelemetryService still combines use-case decisions with JDBC operations, so the domain/infrastructure split is not fully enforced in code.
- **Folder Structure:** Repository tree, two levels deep

Layer	Responsibility	Must Not
Presentation / API	HTTP, WebSocket, validation, auth checks, DTO mapping	Contain business rules or SQL
Application / Service	Use cases, transactions, orchestration	Depend on a specific database or broker
Domain	Entities, value objects, business rules	Import framework or infrastructure code
Infrastructure	Repositories, broker clients, external APIs, caches	Leak vendor types into the domain

6.2 Design Principles Applied

- SOLID: Controllers delegate to TelemetryService and ClickHouseAnalyticsService, keeping HTTP mapping separate from most use cases. The service still owns SQL and some business rules, so dependency inversion and a framework-independent domain layer are incomplete.
- 12-Factor: service addresses, credentials, OIDC issuer, Kafka tuning, and idle thresholds are supplied through environment-backed configuration in Compose; API containers are replaceable and data resides in external volumes. Local example secrets are for developer use only. Stateless horizontal scaling has not been demonstrated.
- Idempotency and validation: event IDs are unique per tenant, late events do not rewind the latest state, request fields are validated, and database ownership uses composite tenant/vehicle foreign keys. Least privilege: hosted API methods check read/ingest/write scopes and tenant claims. Fail-fast: readiness and bounded time-range/limit validation return errors. Rate limits, production service identities, and a complete audit trail are not implemented.

6.3 Design Patterns Used

Pattern	Problem It Solves in Your System	Location in Code
Idempotent consumer / Repository	Duplicate event identity is constrained and persisted once	backend/src/main/ java/com/ fleetintelligence/ telemetry/ TelemetryEventConsumer.java
Retry + dead-letter routing	Bounded processing retries and retained failed events	backend/src/main/ java/com/ fleetintelligence/ telemetry/ TelemetryEventConsumer.java

Pattern	Problem It Solves in Your System	Location in Code
Materialized view / projection	ClickHouse hourly rollups serve analytics; latest vehicle state supports operational reads	backend/src/main/java/com/fleetintelligence/analytics; backend/src/main/resources/clickhouse

6.4 Interfaces, Contracts & Runtime Flows

- **API Contract:** OpenAPI / AsyncAPI link; versioning, pagination, error format, rate limits
- **Event Schemas:** Topics, keys, schema format (Avro / Protobuf / JSON Schema), schema evolution
- **Sequence Diagrams:** Two critical flows, including one failure path (broker down, duplicate event, OEM format change)

Contracts: REST endpoints are versioned under /v1; ingestion returns 202 after Kafka acknowledgement; reads enforce tenant scope when hosted security is enabled. Telemetry is JSON keyed by tenant and vehicle; the local topic is fleet.telemetry.v1 with 12 partitions and seven-day retention. A published OpenAPI/AsyncAPI specification, JSON Schema registry, formal compatibility policy, cursor pagination, and request rate limits are not currently present. Flow A is validation → Kafka acknowledgement → PostgreSQL and ClickHouse consumers → dashboard/report. Flow B is stationary telemetry → persisted warning/critical episode → dashboard acknowledgement → later movement resolves it. Retry/DLQ routing is implemented; broker failure recovery and end-to-end duplicate/replay reconciliation are not verified. Diagrams: Appendix A7–A8.

6.5 Algorithms & Data Structures

- **Problem Solved:** e.g. nearest reachable charger, trip segmentation, VIN / DTC parsing
- **Algorithm Used:** Graph, dynamic programming, regex, streaming structures — justify the choice
- **Pseudocode:** For major functions
- **Time & Space Complexity:** Annotate all
- **Scale Tested:** Input size and measured runtime

Algorithm and pseudocode: deduplicate by (tenant_id,event_id); register ownership; lock one tenant/vehicle state; ignore current-state updates when observed_at is not newer; otherwise update position/speed and stationary_since; open or escalate the idle episode when configured duration thresholds are crossed; resolve it on movement or engine-off; commit. Expected per-event logic is constant work apart from Kafka and indexed database operations (B-tree lookups are $O(\log N)$); one current-state row is maintained per vehicle. The seeded generator is $O(V+E)$ time and $O(1)$ iterator state before output buffering. The Alerts query now uses a matching ordered expression index (expected $O(\log A + \text{page size})$); one local plan sample changed from 12.910 ms scan/sort to 0.100 ms index scan.

Fleet overview aggregates V=100,000 states (82.143 ms in one current plan). Three simulator tests passed, including 100,000 unique IDs; no 100K events/sec runtime test has been run. See docs/ALGORITHMS_AND_CAPACITY.md and evidences/2026-10-02/runtime-and-query-plan-check.md.

7. Non-Functional Requirements & Performance Benchmarks

NFR	Target (Case Study)	Achieved	How Measured
Ingest Throughput	100K+ events/sec	NOT MEASURED	Load test required
End-to-End Latency	< 2 s dashboard; < 5 s critical alert	NOT MEASURED	Timestamped end-to-end test required
API Latency	p95 < 200 ms; p99 < 500 ms	NOT MEASURED	Representative API load test required
Resilience	Recovers after broker / pod failure	NOT VERIFIED	Failure injection and backlog reconciliation required
Availability	99.9%, no single point of failure	NOT VERIFIED	HA deployment and measured availability required

- **Load Test Setup:** Tool, data volume, hardware / cluster size
- **Results:** Graphs or screenshots of throughput, p95 / p99 and consumer lag

Case-study targets: $\geq 100,000$ events/sec; survive 3x burst for five minutes without loss; dashboard freshness < 2 s; critical alert < 5 s; API p95 < 200 ms and p99 < 500 ms; 99.9% availability with no single point of failure. Status: throughput and API/end-to-end percentiles NOT MEASURED; loss and resilience NOT VERIFIED; availability/cloud deployment NOT VERIFIED. The local simulator setting is 5 events/second, not measured capacity. Local observations: 37-hour Reports sample 0.16 s after rollup optimization, current populated ~27-hour analytics sample 0.690 s, and one alert query plan 12.910 ms scan/sort vs 0.100 ms indexed. These are individual local samples, not load-test results. No 100K events/sec or 3x burst run, consumer-lag reconciliation, or soak report exists. Full plan evidence and caveats: evidences/2026-10-02/runtime-and-query-plan-check.md.

8. Security & Compliance

- **Threat Model:** STRIDE for the ingestion path and public API; top five threats and controls
- **Authentication & Authorisation:** OAuth2 / OIDC, RBAC, tenant isolation
- **Device Identity & Encryption:** mTLS, TLS in transit, encryption at rest, secrets management
- **Privacy:** How location and driver data is minimised, masked, retained and erased (GDPR, DPDP Act 2023)

- **AI Safety (if applicable):** Prompt-injection defences, tool permissions, human approval, audit log

Threat model: see docs/SECURITY_THREAT_MODEL.md for the STRIDE review. Spoofing: hosted API mode validates OIDC JWT signature/issuer/expiry and scopes; device identity/mTLS is absent. Tampering: validation, idempotency, and tenant-scoped relational constraints exist; TLS and formal schema evolution are absent. Repudiation: acknowledgement timestamp/operator fields exist, but an append-only access/action audit log does not. Information disclosure: tenant claim/query checks exist in hosted mode; privacy masking and erasure are not implemented. Denial of service: Kafka buffers accepted events and retries are bounded, but rate limits/quotas and capacity controls are absent. Elevation of privilege: scope/tenant checks exist; no production role/provider configuration has been verified. Local Compose disables API authentication for the demo. No security scan or penetration test has been performed. Seven-day Kafka retention and 90-day ClickHouse raw retention are configured; PostgreSQL erasure and cold archive are not. Only synthetic data is intended; this prototype does not claim GDPR/DPDP compliance.

9. Test Strategy

Test Type	Tools	No. of Tests	Coverage / Result	In CI?
Unit	Python unittest	3	Passed locally; coverage percentage not measured	Yes
Integration & Contract	Spring Boot + Testcontainers	10 integration methods	Earlier CI covers Kafka/PostgreSQL; ClickHouse is mocked. V7 CI rerun pending.	Integration yes; separate contract suite no
Acceptance (BDD)	Not configured	0 reported	No BDD scenarios recorded	No
Performance / Load / Soak	No controlled tool run	0 target-scale runs	100K events/sec, 3x burst, loss, lag and p95/p99 not measured	No
Security (SAST, DAST, Dependency, Image)	No consolidated scans	0 reports	No verified scan report	No
Compliance & Chaos	Manual design only	0 reported	Audit/erasure and broker/pod recovery not verified	No

- **Edge Cases Covered:** e.g. duplicate events, out-of-order data, unknown OEM format, broker outage
- **Sample Input / Output:** If helpful
- **Report Links:** Coverage, load-test and security scan reports

10. Observability

- **Dashboard Screenshot:** Throughput, consumer lag, error rate, latency percentiles
- **Troubleshooting Walk-Through:** How you would find the cause of a latency spike using logs, metrics and traces

Verified checks: three simulator unit tests passed locally; the React/TypeScript production build passed; the API Docker image packaged successfully; API readiness, dashboard, overview, vehicles, alerts, and analytics endpoints returned HTTP 200; and current local query plans are recorded. Earlier GitHub Actions passed a ten-method Kafka/PostgreSQL Testcontainers integration suite; the V7 migration requires its own CI run after push. No aggregate line/branch coverage percentage, separate contract suite, BDD acceptance suite, target-scale load/soak, security scan, chaos/failure recovery, or observability dashboard screenshot is available. For a latency spike, compare request rate and API p95/p99 with Kafka consumer lag and retry/DLQ counts; correlate API logs/request IDs with broker and database timeouts; use distributed traces to locate the slow hop. Metrics/traces are not yet instrumented, so this is a runbook plan, not a demonstrated walkthrough.

11. AI / ML Component (if used)

- **Purpose:** The decision the model or agent supports, and why rules alone were not enough
- **Data & Features:** Training or retrieval data, how it was generated or sourced, leakage checks
- **Model / Agent Design:** Model choice, vector store, tools the agent can call, how access is limited
- **Evaluation:** Metrics against a simple baseline
- **Guardrails & Cost:** Failure handling, hallucination checks, human-in-the-loop, cost and latency per request

No machine-learning model, vector retrieval, or LLM agent is part of the running product. Idle alerts use deterministic, explainable threshold rules; no trained-model use case or evaluation dataset exists. The brief makes vector storage conditional ("where useful"), so no vector store is installed without a grounded retrieval need. If a model is added later, evaluate it against the current rules baseline and include guardrails, human approval, access limits, latency, cost, and audit evidence.

12. Architecture Decisions, Risks & Future Enhancements

- **Architecture Decision Records:** 3–5 records: Context → Options Considered → Decision → Consequences (cover databases, messaging and one CAP trade-off)

- **Risks & Technical Debt:** Known limitations and shortcuts taken during the hackathon
- **Future Enhancements:** Next three steps from prototype to pilot

Four ADRs record the product scope, Kafka messaging, ClickHouse analytics, and consistency/retention decisions (docs/adr/0001 through 0004). Risks include the single-node local topology, synthetic-only telemetry, estimated fuel/emissions, incomplete security controls, and unmeasured challenge-scale behavior. Next steps are to add cloud/HA deployment and production security settings; run controlled load, latency, coverage, security, and failure tests on suitable infrastructure; and record the five-minute demo before tagging a verified release. Current local observations do not establish production impact.

13. Demo Video (5 Minutes Maximum)

Videos longer than 5:00 are cut at 5:00. Show the working product, not only slides. 1080p, clear audio, captions recommended; unlisted YouTube, Vimeo or Drive link.

Time	Segment	What to Show
0:00 – 0:30	Problem	The user, the pain, and one number that proves it matters
0:30 – 1:00	Solution	One-line pitch and key value proposition
1:00 – 3:00	Live Demo	Main user journey end to end on live data, including one real-time event
3:00 – 4:15	Under the Hood	Architecture, load test or dashboard at scale, one failure-recovery moment
4:15 – 5:00	Impact & Next Steps	Measured results, what comes next, and the team

- Video Link: **[ADD UNLISTED VIDEO URL — MAXIMUM 5 MINUTES]**

14. Repository Checklist

- **README:** Problem, architecture image, quick start, environment variables, test commands, known issues
- **One-Command Run:** docker compose up (or make run) starts the full stack with the data simulator
- **Structure:** Clear service folders, plus /docs, /infra and /tests
- **CI Pipeline:** Build, lint, all test suites and security scans on every push
- **Hygiene:** No secrets committed, .env.example provided, commits from all members
- **Final Tag:** v1.0-submission

15. Conclusion

- Summary of key learnings

- Strengths of the solution
- Challenges tackled and solved

The prototype connects synthetic telemetry to operational fleet monitoring and historical analytics. Its clearest strength is an explainable path from event to alert and dashboard, with a separately queryable historical store. The main lesson is that functional correctness and local query optimizations do not prove the stated throughput, latency, resilience, security, or availability targets. The next phase is to validate these on a suitable multi-node environment, close production controls, and record the live demo. [TEAM: add your own learning and final team reflection after validation.]

16. Declarations

- **Open-Source Components:** List with licences (an SBOM file in the repository is acceptable)
- **AI Tools Used:** Coding assistants or LLM APIs, and the parts of the work they were used for
- **Data:** Confirm all data is synthetic or public and contains no real personal data

Open-source components and licences: direct dependencies and container base images are declared in backend/pom.xml, frontend/package-lock.json, Dockerfiles, and Compose. A complete transitive SBOM/license report has not yet been generated; review it before submission. AI tools: OpenAI Codex assisted with code review, implementation, and document drafting; no LLM API is called by the running product. Data declaration: simulator and demo records are synthetic; fuel and CO2 figures are estimates, not sensor measurements. [TEAM: list any other tools used and confirm the final data/formal declaration.]

17. Appendix (if any)

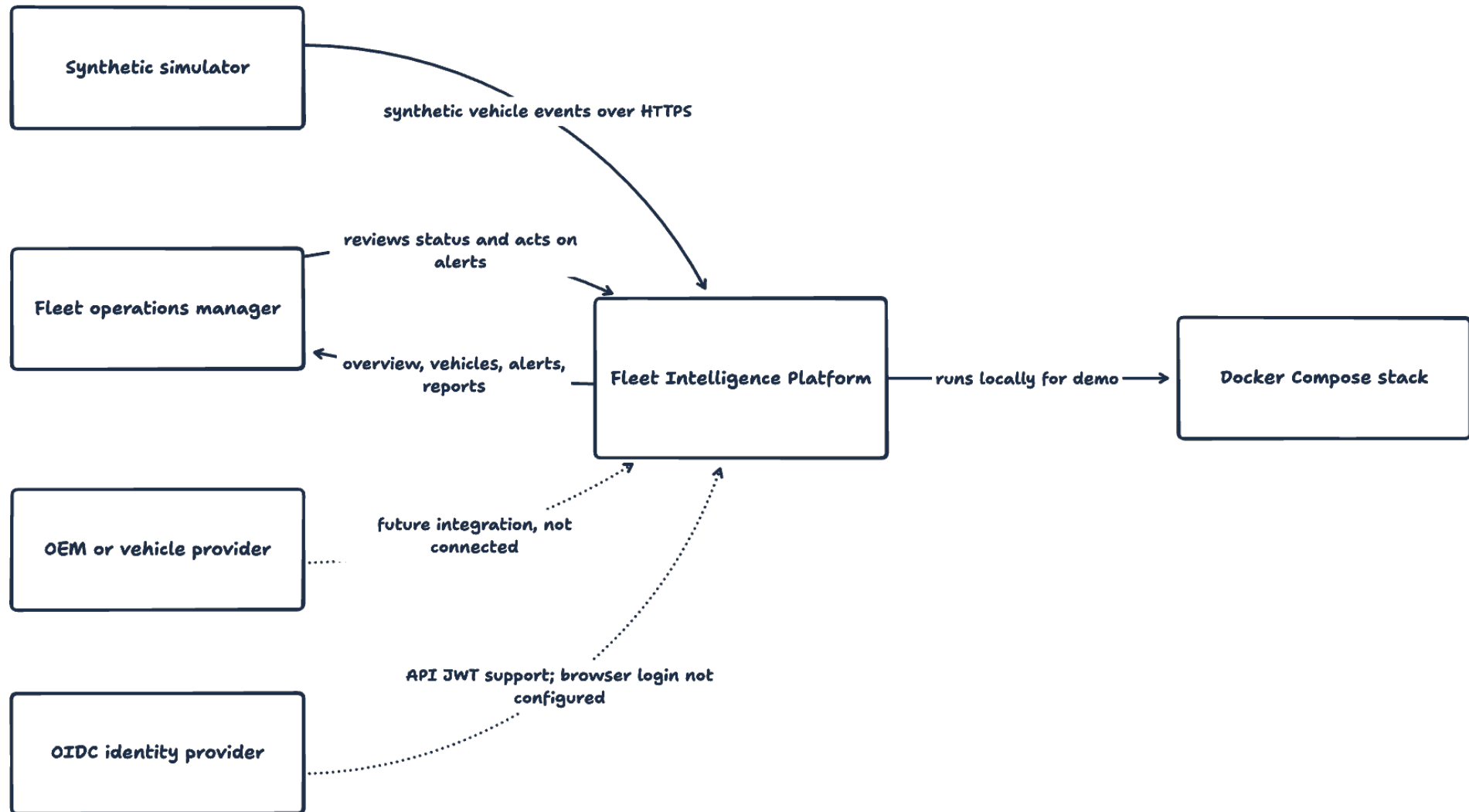
- Additional architecture, deployment, ER, layering, and sequence diagrams: see landscape Appendix A following the template pages.
- Full test logs or large datasets used
- References or external sources

Appendix A — Architecture and Data Diagrams

Editable source: tldraw board “Fleet Intelligence Platform — Hackathon Architecture & Data Diagrams”. Implemented components reflect the local demo. Any future integrations or high-availability topology are explicitly marked as planned, not deployed. Latency figures are targets only until measured.

A1. C4 system context

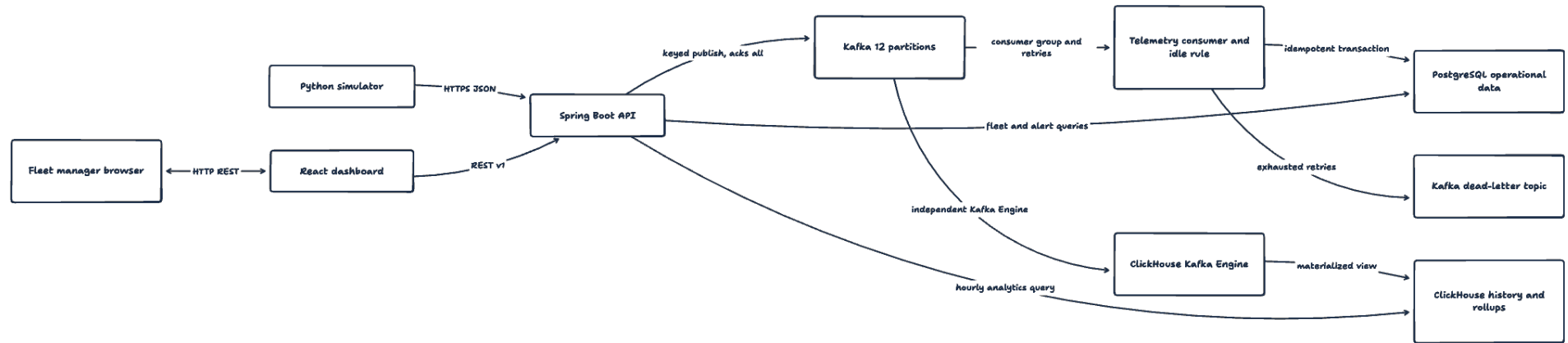
01 C4 Context | Fleet Intelligence Platform



Fleet manager, simulator, platform boundary, and external integrations not connected in the demo.

A2. C4 containers

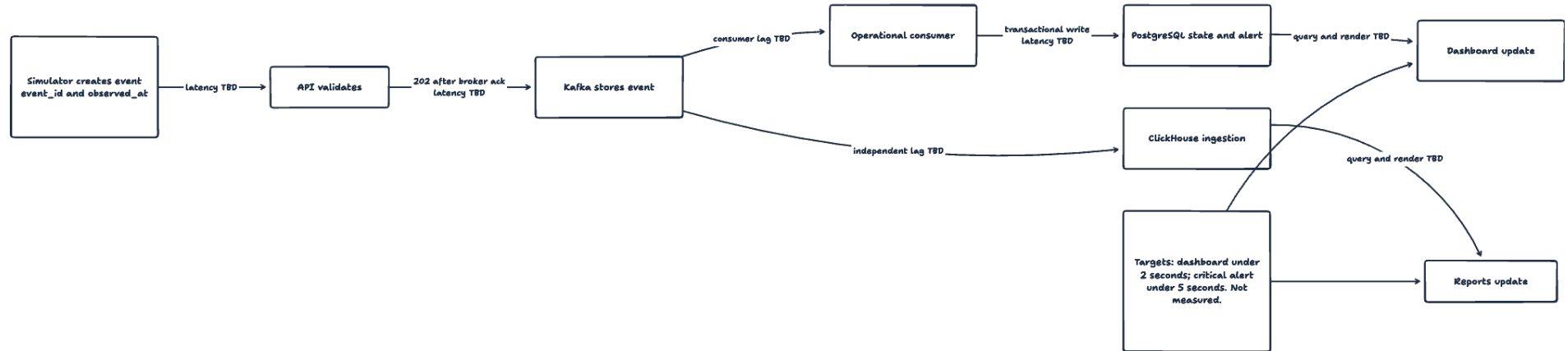
02 C4 Containers | Current local runtime



Current services, stores, protocols, event routes, and dead-letter handling.

A3. Telemetry event flow

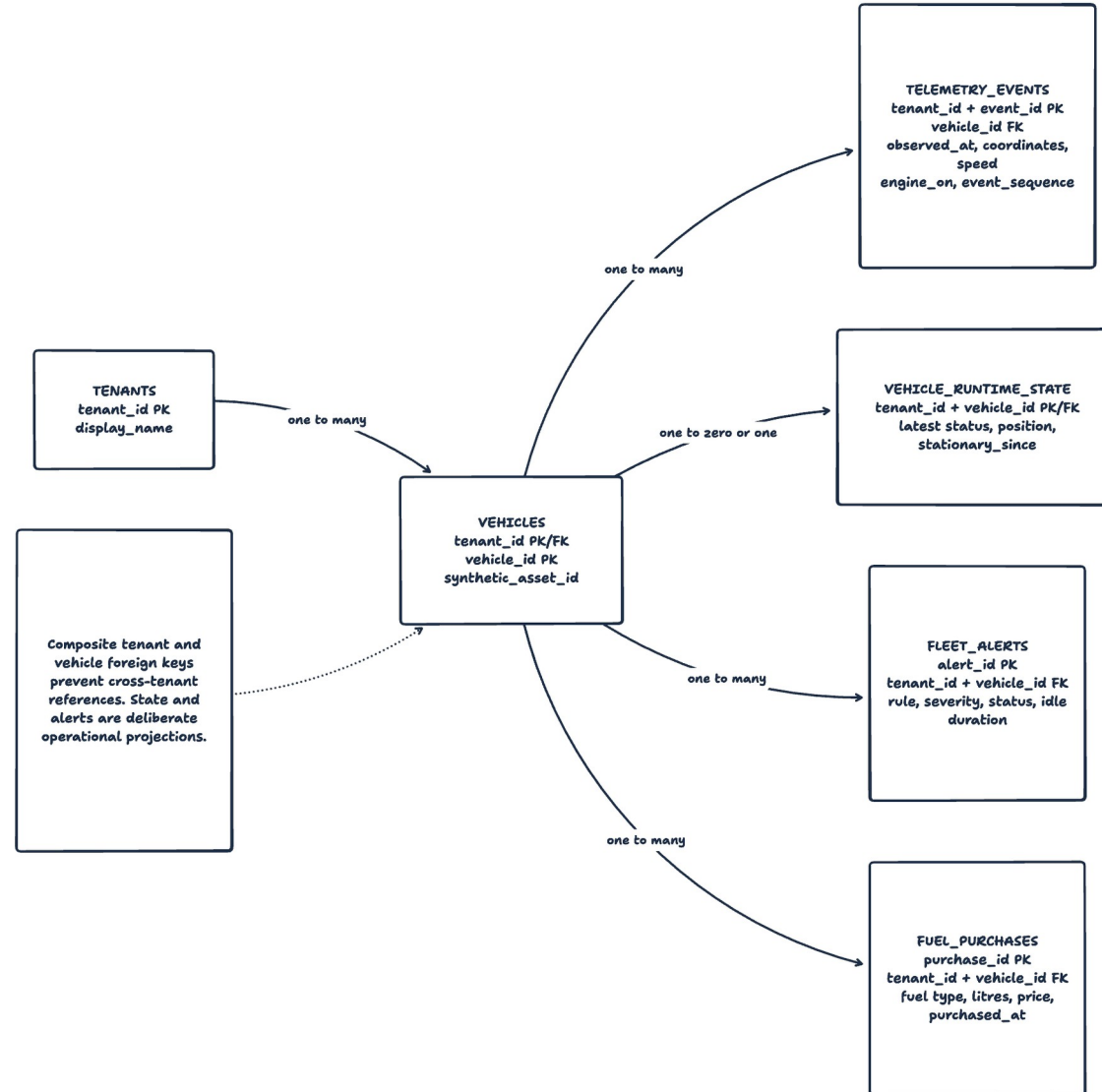
03 Event Data Flow | Per-hop latency TBD



Per-hop latency is marked **TBD**; dashboard and critical-alert values are challenge targets until measured.

A4. Relational ER and tenant ownership

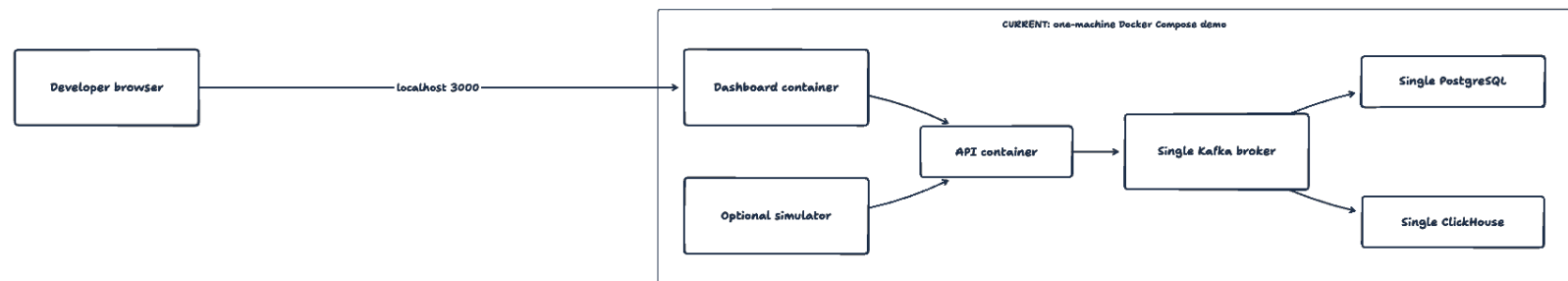
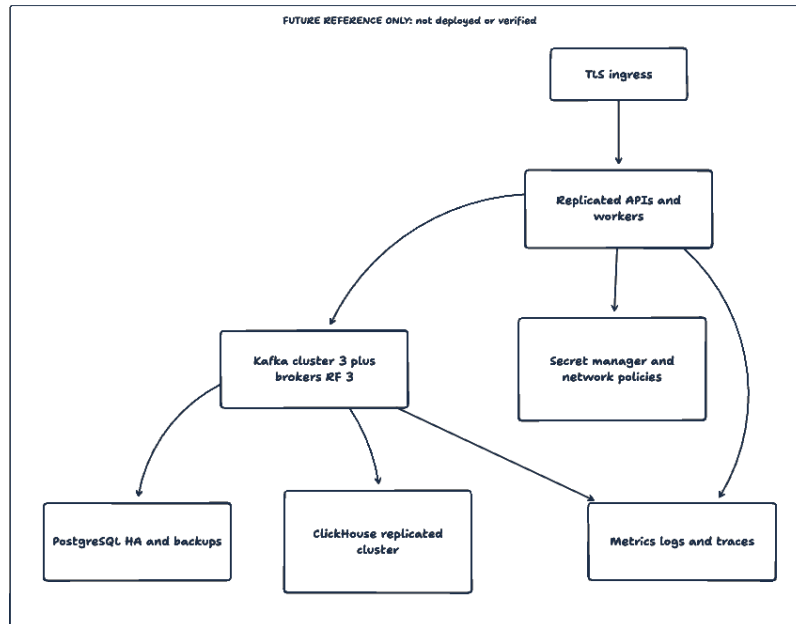
04 ER and Tenant Ownership | Post V6



Post-V6 tenant and vehicle ownership, operational projections, and telemetry/fuel relationships.

A5. Deployment view

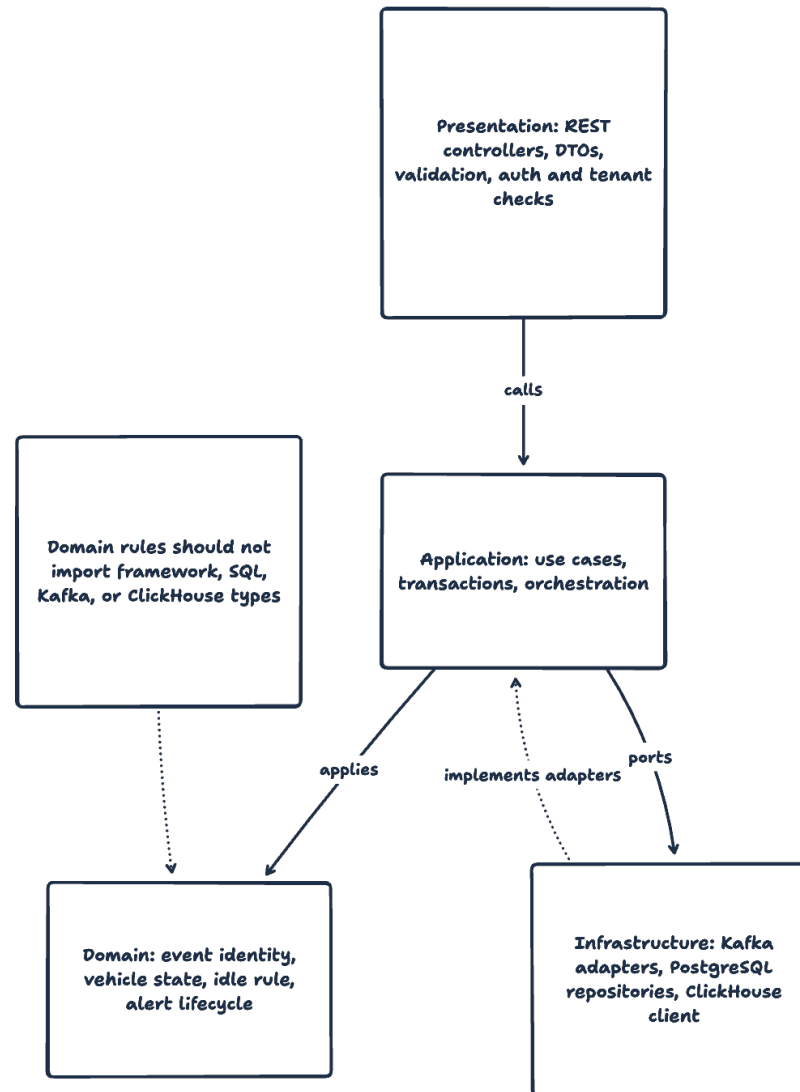
OS Deployment | Demo now and HA target later



Current local Compose topology beside a proposed HA target that is not deployed or verified.

A6. Service layers

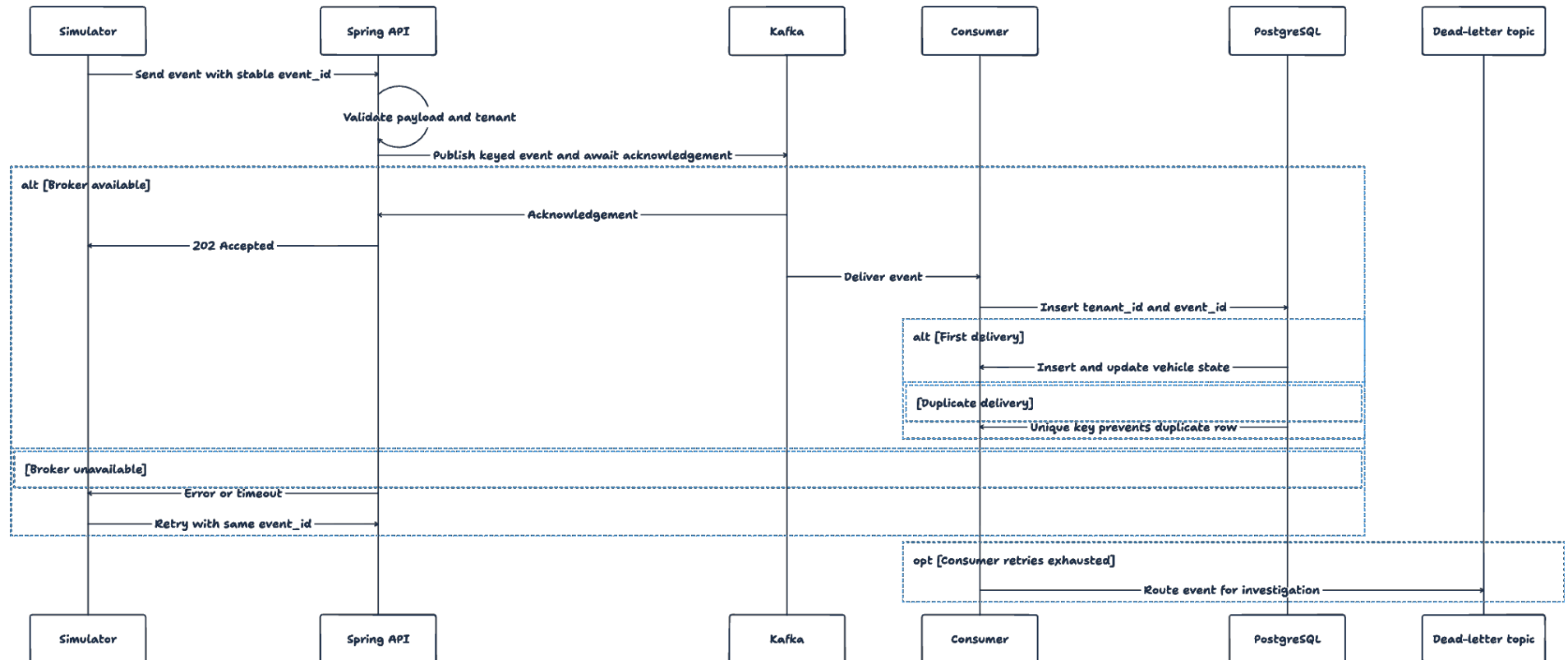
06 Service Layers | Intended dependency direction



Intended presentation, application, domain, and infrastructure responsibilities.

A7. Sequence: ingest and failure handling

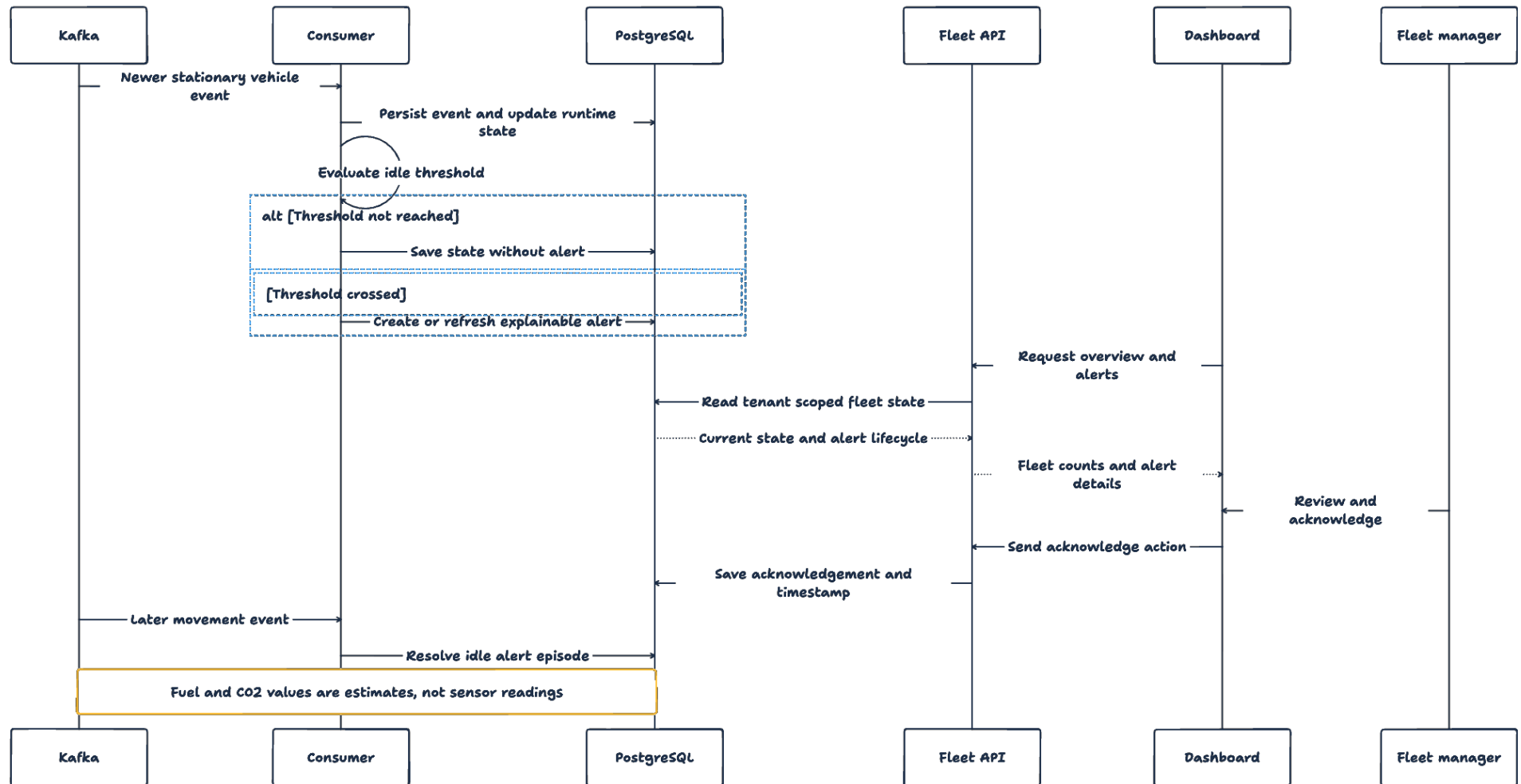
07 Sequence | Ingest, duplicate, and broker failure



Kafka acknowledgement, duplicate idempotency, broker unavailable retry, and dead-letter path.

A8. Sequence: idle alert lifecycle

08 Sequence | Idle alert, acknowledgement, resolution



Idle threshold, persisted alert, dashboard acknowledgement, and later movement resolution.